

Detección de vallas publicitarias con Transformers: pre-etiquetado con Grounding DINO y ajuste fino de RT-DETR

Proyecto final · Visión por Computadora III

Christian Conchari

Maestría en Inteligencia Artificial, CEIA, FIUBA · Octubre de 2026

Resumen

Este trabajo desarrolla un detector de vallas publicitarias de vía pública para fotogramas de video grabados desde un vehículo, a partir de 632 imágenes sin etiquetas confiables. Se emplea un esquema en dos etapas: Grounding DINO, un detector de vocabulario abierto, propone cajas que luego se corrigen manualmente, y con ese conjunto se ajusta RT-DETR, un detector Transformer de tiempo real. Sobre un conjunto de prueba etiquetado desde cero, RT-DETR alcanza 0.649 de mAP y 0.930 de AP50, frente a 0.481 y 0.723 de Grounding DINO sin ajuste, y procesa 51 imágenes por segundo, frente a 2.1 de Grounding DINO en la configuración usada. Agregar fotogramas de los mismos videos deja de mejorar el resultado a partir de unas 200 imágenes revisadas. El análisis del error sugiere que la pérdida de mAP a IoU alto no se debe a vallas no detectadas, sino a un sesgo en el tamaño de las cajas, consistente con el estilo de las pre-etiquetas.

Repositorio: <https://github.com/ChristianConchari/vit-billboard-detector>

1. Objetivo del proyecto

El objetivo es detectar vallas publicitarias de vía pública (*out-of-home*, OOH) en fotogramas de video grabados desde un vehículo, con un detector basado en Transformers capaz de operar en tiempo real. La aplicación prevista es la auditoría de publicidad exterior: inventariar las vallas de un recorrido y verificar su presencia o su exposición, lo que exige procesar video con hardware accesible.

La restricción principal son los datos: 632 fotogramas de 26 videos, sin etiquetas y con poco tiempo para anotarlos. Esta restricción pesa más sobre un Transformer (Vaswani et al., 2017) que sobre una CNN. Al carecer del sesgo inductivo de la convolución (localidad e invariancia a traslaciones), los ViT aprenden peor con pocos datos y solo superan a las CNN cuando se preentrenan a gran escala (Dosovitskiy et al., 2021; Zhu, H. et al., 2023). RT-DETR atenúa este problema porque conserva un backbone convolucional y aplica la atención solo en el encoder y el decoder. Aun así, surgen dos decisiones: partir de pesos preentrenados en lugar de entrenar desde cero, y abaratar el etiquetado sin sesgar la evaluación.

Objetivos específicos:

1. Construir el conjunto de datos corrigiendo las propuestas de un modelo *zero-shot*, en lugar de dibujar cada caja.
2. Ajustar RT-DETR y compararlo con Grounding DINO *zero-shot* sobre un conjunto de prueba cuyo *ground truth* sea independiente de ambos modelos.
3. Medir cuántas imágenes etiquetadas hacen falta y dónde se pierde precisión.
4. Entregar un sistema de nivel preproductivo: *pipeline* reproducible, registro de experimentos, pruebas automatizadas y una herramienta de inferencia sobre imágenes y video.

2. Arquitectura general

2.1 Flujo del sistema

El sistema tiene dos fases (figura 1): la construcción del conjunto de datos (etapa 1), en la que una persona revisa las propuestas de Grounding DINO, y el entrenamiento y la evaluación (etapas 2 a 4), que se ejecutan con un único comando. Los videos se asignan a las particiones antes de etiquetar, de modo que los de prueba se anotan sin pre-etiquetas desde el principio.

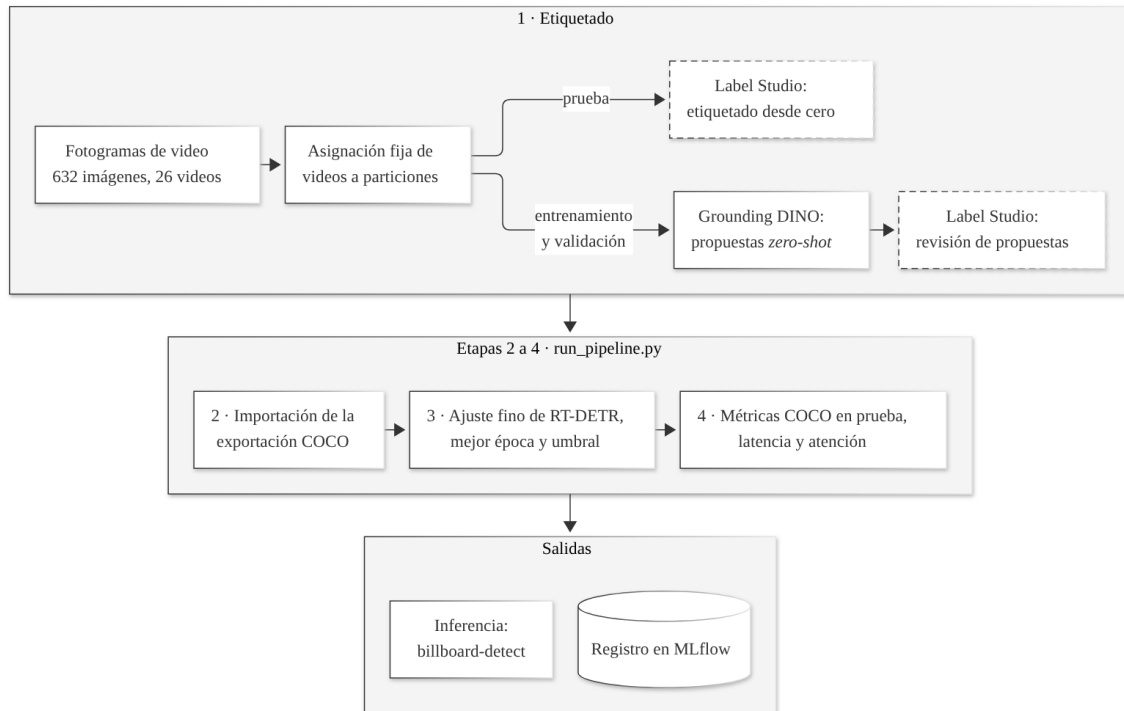


Figura 1. Flujo del sistema, desde los fotogramas hasta el modelo calibrado. Los recuadros con borde discontinuo son pasos manuales.

Etapa		Componente	Función
1	Asignación por video		Asigna cada video completo a entrenamiento, validación o prueba, una sola vez y antes de etiquetar
1	Pre-etiquetado		Grounding DINO con tres <i>prompts</i> ; las cajas duplicadas entre <i>prompts</i> se eliminan con NMS
1	Revisión y etiquetado		En Label Studio: corrección de propuestas en entrenamiento y validación; etiquetado desde cero en prueba
2	Importación		Lee la exportación COCO de Label Studio, separa las particiones y registra una huella del contenido de cada una
3	Entrenamiento y calibración		Ajusta RT-DETR, conserva la mejor época de validación y fija el umbral de confianza
4	Evaluación		Mide ambos modelos con el mismo código sobre la prueba, junto con la latencia y los mapas de atención
—	Inferencia		Comando <code>billboard-detect</code> para imágenes, carpetas y video, con el umbral calibrado
—	Registro		Cada ejecución de las etapas 2 a 4 queda registrada en MLflow

El diagnóstico de localización (sección 5.5) se calcula con un script aparte sobre el modelo final.

2.2 Modelos

RT-DETR. DETR (Carion et al., 2020) formula la detección como una predicción de conjuntos: el decoder emite un número fijo de predicciones y, durante el entrenamiento, cada objeto se asigna a una sola de ellas mediante un emparejamiento bipartito. Como cada objeto tiene una única predicción responsable, no hace falta NMS. Sus limitaciones son el costo de la atención sobre todo el mapa de características y una convergencia lenta, de unas 500 épocas en COCO. RT-DETR (Zhao et al., 2024) conserva esa formulación y reduce el costo del encoder (figura 2).

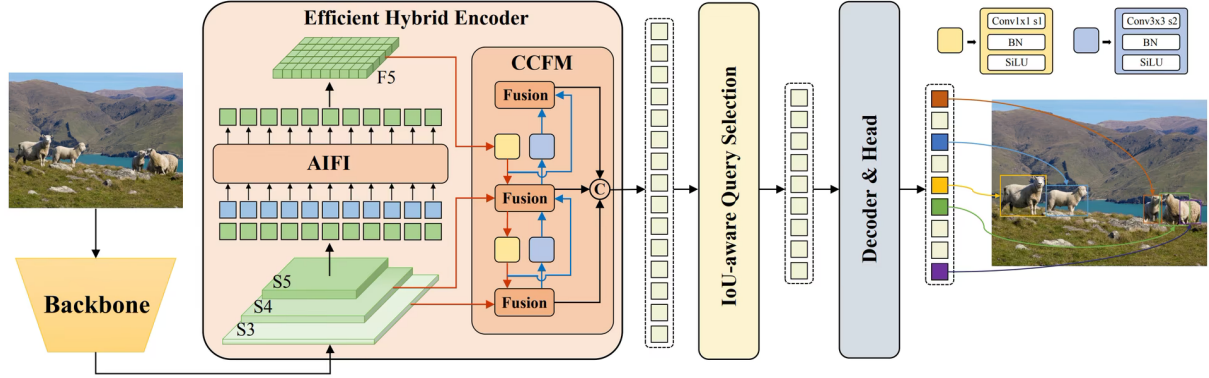


Figura 2. Arquitectura de RT-DETR. Reproducida de Zhao et al. (2024), figura 3 de la versión arXiv:2304.08069v2. En este trabajo, el backbone es una ResNet-50 y la cabeza predice una sola clase.

- **Encoder híbrido** (*Efficient Hybrid Encoder*). El backbone entrega tres mapas de características (S3, S4 y S5). La *self-attention* del módulo AIFI opera solo sobre S5, el de menor resolución (20×20 tokens para una entrada de 640×640), y produce F5. El módulo CCFM fusiona F5 con S4 y S3 mediante bloques convolucionales y concatena el resultado.
- **Selección de *queries* guiada por IoU** (*IoU-aware Query Selection*). Los 300 tokens del encoder con mayor puntaje, entrenado para reflejar la calidad de localización, inician las *queries* del decoder, lo que acelera la convergencia.
- **Decoder y cabezas** (*Decoder & Head*). Cada capa del decoder combina *self-attention* entre las *queries*, que ayuda a evitar predicciones duplicadas, con *cross-attention* deformable sobre los mapas del encoder (Zhu, X. et al., 2021): cada *query* muestrea 96 ubicaciones ($8 \text{ cabezas} \times 3 \text{ escalas} \times 4 \text{ puntos}$) en lugar de atender a toda la imagen. Las cabezas predicen la caja y la clase de cada *query*. Durante el entrenamiento se agregan *queries* de *denoising* contrastivo, construidas a partir del *ground truth* con ruido, que estabilizan el emparejamiento.

La variante usada está preentrenada en COCO; sus cabezas de clasificación se reinician de 80 clases a una sola (*billboard*).

Grounding DINO. Es un detector de la familia DETR que recibe una imagen y un texto (Liu et al., 2024). Codifica la imagen con Swin-T y el texto con BERT, y fusiona ambas modalidades para asociar cada caja a una frase del *prompt*. Esto le permite detectar categorías arbitrarias sin reentrenamiento, lo que lo hace adecuado para pre-etiquetar. Su desventaja es la velocidad. Además, el *pipeline zero-shot-object-detection* de Hugging Face, usado en este trabajo, ejecuta una pasada por cada *prompt*, aunque el modelo admite varias frases en una sola pasada; las cajas duplicadas entre *prompts* se eliminan luego con NMS, tanto al pre-etiquetar como al evaluar. En este proyecto cumple dos funciones: etiquetador *offline* y línea de base *zero-shot*.

3. Implementación técnica

3.1 Herramientas

Área	Herramienta
Modelos	Hugging Face <code>transformers</code> 5.17 (PekingU/ <code>rtdetr_r50vd</code> , IDEA-Research/ <code>grounding-dino-tiny</code>), PyTorch 2.14
Etiquetado	Label Studio 1.23
Evaluación	<code>pycocotools</code> (COCOeval)
Registro de experimentos	MLflow 3.16
Imágenes y video	OpenCV, Pillow, <code>torchvision.transforms.v2</code>
Calidad	<code>pytest</code> (112 pruebas), <code>ruff</code> , GitHub Actions
Hardware	NVIDIA RTX 4070 SUPER (12 GB), Intel Core i7-12700F, 16 GB de RAM

3.2 Construcción del conjunto de datos

1. **Descarte de las etiquetas existentes.** El conjunto original incluía anotaciones, pero el 72 % no había sido revisado y algunas cajas rechazadas figuraban como positivas (ADR 0001).
2. **Pre-etiquetado.** Grounding DINO, con los *prompts* “billboard”, “advertising sign” y “ooh ad” y un umbral de 0.35, generó 1194 propuestas sobre las 632 imágenes.
3. **Revisión.** Las propuestas se corrigieron, eliminaron o completaron en Label Studio. Se revisaron 631 de las 632 imágenes; la restante, perteneciente a entrenamiento, quedó fuera del conjunto. Las imágenes sin vallas se conservaron como ejemplos negativos.
4. **Partición por video** (ADR 0002). Los fotogramas consecutivos de un video son casi idénticos, de modo que una partición aleatoria por imagen filtraría duplicados del entrenamiento a la prueba. Cada video se asigna completo a una partición (70/15/15 en imágenes), y la asignación se mantiene fija durante el etiquetado incremental.
5. **Prueba etiquetada desde cero** (ADR 0003; sección 4.2).

Partición	Videos	Imágenes	Cajas	Negativos
Entrenamiento	12	441	798	3
Validación	7	95	130	1
Prueba	7	95	126	1

3.3 Entrenamiento

- **Optimización.** AdamW (tasa de aprendizaje $1e-4$, *weight decay* $1e-4$), decaimiento coseno de la tasa de aprendizaje, recorte de gradiente en 0.1, lotes de 8 imágenes y 50 épocas, con entradas de 640×640 . Si el backbone se descongela, usa una tasa diez veces menor ($1e-5$).
- **Aumentación.** Volteo horizontal ($p = 0.5$), escala entre 0.75 y 1.25, traslación del 10 % y variación de color de 0.2, aplicando a las cajas las mismas transformaciones que a la imagen.
- **Inicialización del sesgo de clase.** Al reinicializar la cabeza, `transformers` usa un prior de $1/(\text{clases} + 1)$, que con una sola clase da 0.5 y deja el sesgo en cero: las 300 *queries* parten con probabilidad 0.5 y la pérdida total inicial (clasificación, L1 y GIoU, sumadas sobre las capas del decoder) llega a 287. Con el prior de RT-DETR ($p = 0.01$, sesgo ≈ -4.6) baja a 40.
- **Selección del checkpoint.** Se conserva la época con mejor mAP de validación.

- **Calibración del umbral.** En validación, cada predicción se empareja con el *ground truth* ($\text{IoU} \geq 0.5$) y se elige el umbral de confianza que maximiza F1. En el modelo final, ese umbral es 0.15 y da una precisión de 0.85 y un *recall* de 0.93 en validación. El umbral se guarda con el checkpoint y la herramienta de inferencia lo usa por defecto. Se calibra para cada checkpoint porque la escala de la confianza varía entre entrenamientos.

3.4 Módulos clave

El código es una biblioteca instalable (`src/vit/`) organizada por etapa. Los scripts de `scripts/` solo leen argumentos y configuración e invocan a la biblioteca, de modo que el *pipeline* y los scripts comparten el mismo código.

Paquete	Contenido
<code>labeling/</code>	Pre-etiquetado con Grounding DINO y carga de pre-annotaciones en Label Studio
<code>data/</code>	Importación de anotaciones, partición por video, huella de contenido y conjunto COCO para entrenamiento
<code>models/</code> , <code>transforms/</code> , <code>train/</code>	Carga y adaptación de RT-DETR, aumentación y bucle de entrenamiento
<code>inference/</code>	Interfaz común <code>Detector</code> , adaptadores de ambos modelos, inferencia sobre imágenes y video, descarga verificada de pesos
<code>eval/</code>	Métricas COCO, calibración, latencia, localización, atención y gráficos
<code>utils/</code>	Configuración, <i>logging</i> , registro en MLflow y selección del dispositivo
<code>pipeline.py</code>	Orquestación del proceso completo
<code>src/interfaces/cli/</code>	Comando <code>billboard-detect</code>

Tres decisiones de diseño sostienen la validez de los resultados:

1. **Una interfaz común para ambos detectores.** RT-DETR y Grounding DINO devuelven el mismo tipo de predicción y se evalúan con el mismo código, así que ninguna diferencia de métricas puede atribuirse a la forma de evaluar.
2. **Una corrida de MLflow por ejecución.** Entrenamiento, evaluación y artefactos quedan en un único registro; el entrenamiento falla si no hay una corrida activa.
3. **Huella de contenido de cada partición.** Se calcula a partir de los archivos y las cajas, no de los identificadores que Label Studio renumera en cada exportación. Esto permite rechazar comparaciones entre corridas evaluadas sobre conjuntos de prueba distintos.

3.5 Reproducibilidad

Cada ejecución registra en MLflow el commit, si había cambios sin confirmar, la huella de cada partición, la configuración completa, las métricas por época y finales, y los reportes generados (figura 3). El registro se exporta a archivos versionados, de modo que los resultados pueden verificarse sin la base local. Los pesos del modelo final se publican como versión del repositorio (*release*) y se descargan verificando su suma SHA-256. La configuración por defecto corresponde al backbone congelado; el modelo final se reproduce con:

```
python scripts/run_pipeline.py --export-path <exportación>.zip \
--set training.freeze_backbone=false --set training.seed=2
```

	Run Name	Created	Duration	Source	Metrics			Parameters			Tags
					test/rtdestr/AP5	test/rtdestr/mAI	val/best_mAP	rtdestr.training.f	rtdestr.training.s	train.images	
☐	rtdestr-20261005-004038	1 hour ago	23.0min	run_pip...	0.93009792...	0.64863236...	0.81679133...	False	2	441	ablation
☐	rtdestr-20261005-001117	1 hour ago	23.0min	run_pip...	0.93753093...	0.64888132...	0.78716085...	False	1	441	ablation
☐	rtdestr-20261004-235251	1 hour ago	18.1min	run_pip...	0.93285973...	0.65257130...	0.80164738...	True	2	441	ablation
☐	rtdestr-20261004-232152	2 hours ago	18.3min	run_pip...	0.92513021...	0.63799412...	0.79509731...	True	1	441	ablation
☐	rtdestr-20261004-225320	2 hours ago	23.9min	run_pip...	0.93408875...	0.64512628...	0.77555178...	False	42	441	ablation
☐	rtdestr-20261004-221908	3 hours ago	18.9min	run_pip...	0.92764615...	0.63719620...	0.74815723...	True	42	441	learning-cu...
☐	rtdestr-20261004-204025	5 hours ago	16.6min	run_pip...	0.92710567...	0.64193623...	0.76411410...	True	42	346	learning-cu...
☐	rtdestr-20261004-200415	5 hours ago	12.5min	run_pip...	0.92159225...	0.63717949...	0.75820642...	True	42	205	learning-cu...
☐	rtdestr-20261001-232751	3 days ago	8.0min	run_pip...	0.78418994...	0.54617707...	0.70180867...	True	42	66	learning-cu...

Figura 3. Corridas de la curva de aprendizaje y de la ablación registradas en MLflow, con sus parámetros y métricas.

4. Evaluación

4.1 Métricas

- **mAP@[.5:.95]** (Lin et al., 2014): AP promedio en 10 umbrales de IoU (0.50 a 0.95, paso 0.05), con 101 puntos de *recall* y un máximo de 100 detecciones por imagen. Exige cajas muy ajustadas.
- **AP50 y AP75**: AP a IoU 0.5, que tolera cajas imprecisas, y a IoU 0.75, que exige una localización ajustada. El AP es el área bajo la curva de precisión y *recall*, no la proporción de vallas detectadas.
- **Latencia**: tiempo de punta a punta (preprocesamiento, modelo y postprocesamiento) por imagen, de a una, en GPU.

La evaluación ignora la clase, porque hay una sola y Grounding DINO etiqueta sus cajas con el texto del *prompt*. Para el mAP se usa un umbral de confianza bajo, ya que la métrica integra toda la curva de precisión y *recall*. La diferencia entre modelos (0.01 en RT-DETR y 0.05 en Grounding DINO) proviene de la configuración inicial y, si tiene algún efecto, desfavorece levemente a Grounding DINO. La precisión y el *recall* en el punto de operación (umbral calibrado de 0.15) se midieron solo en validación. El AP por tamaño de objeto no se reporta: 125 de las 126 cajas de prueba son grandes.

4.2 Un conjunto de prueba independiente

El primer conjunto de prueba se construyó con las mismas pre-etiquetas corregidas. El 67% de sus cajas coincidía con las de Grounding DINO ($\text{IoU} > 0.98$), lo que elevaba el mAP de ese modelo a 0.677. Para eliminar el sesgo, los 7 videos de prueba (95 imágenes) se etiquetaron a mano sin pre-etiquetas, y ninguna caja resultante coincide con una propuesta (IoU máximo de 0.974). El conjunto quedó fijo desde entonces.

4.3 Protocolo experimental

1. **Curva de aprendizaje.** Misma configuración con 66, 205, 346 y 441 imágenes de entrenamiento.
2. **Ablación del backbone.** Congelado y descongelado, con tres semillas cada uno; se reportan la media y el rango.
3. **Diagnóstico de localización.** AP por umbral de IoU y sesgo de los bordes de las cajas.
4. **Selección del modelo final.** El checkpoint con mejor mAP de validación entre las seis corridas de la ablación (backbone descongelado, semilla 2), elegido sin consultar la prueba.

5. Resultados y ejemplos

5.1 Comparación de modelos

Modelo	mAP@[.5:.95]	AP50	AP75
RT-DETR ajustado (modelo final)	0.649	0.930	0.794
RT-DETR ajustado, media (rango) de 3 semillas	0.648 (0.645–0.649)	0.934 (0.930–0.938)	0.783 (0.777–0.794)
Grounding DINO <i>zero-shot</i>	0.481	0.723	0.588

Sobre las 95 imágenes de prueba, RT-DETR supera a Grounding DINO por 0.17 de mAP y 0.21 de AP50, y alcanza un AP50 de 0.930 frente a 0.723. La variación entre semillas es pequeña frente a esa diferencia.

5.2 Curva de aprendizaje



Figura 4. mAP de prueba de RT-DETR según el número de imágenes de entrenamiento; la línea horizontal corresponde a Grounding DINO.

Con solo 66 imágenes, RT-DETR ya supera a Grounding DINO (0.546 frente a 0.481 de mAP). Con 205 alcanza 0.637 y, a partir de ahí, el mAP se estabiliza: con 441 imágenes sigue en 0.637, una variación menor que la dispersión entre semillas (0.015). Como todas las imágenes provienen de los mismos 12 videos, el resultado mide la redundancia entre fotogramas de un mismo video, no cuántas escenas distintas harían falta.

5.3 Ablación del backbone y dinámica del entrenamiento

Backbone	mAP	AP50	AP75
Congelado (3 semillas)	0.643 (0.637–0.653)	0.929 (0.925–0.933)	0.784 (0.778–0.794)
Descongelado (3 semillas)	0.648 (0.645–0.649)	0.934 (0.930–0.938)	0.783 (0.777–0.794)

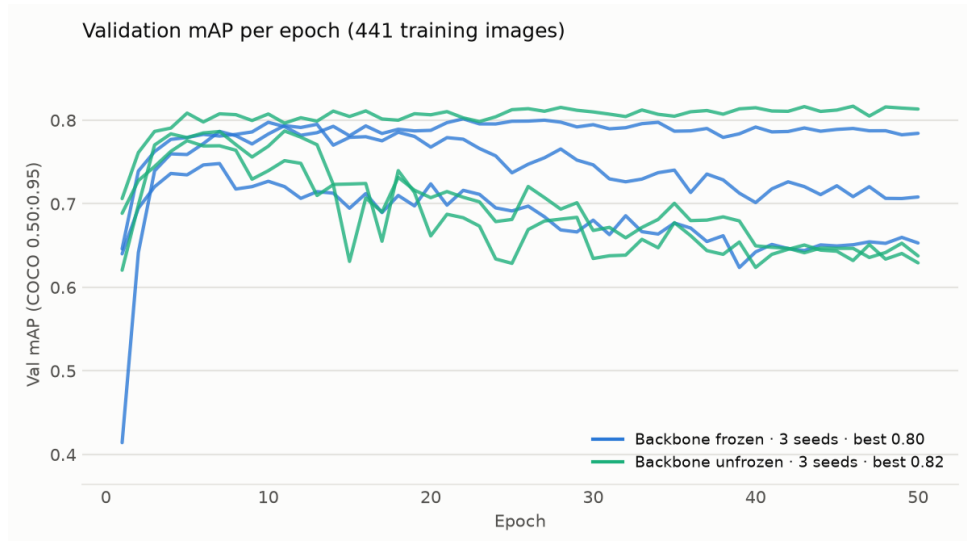


Figura 5. mAP de validación por época en las seis corridas de la ablación.

Descongelar el backbone no mejora la precisión de forma medible: la diferencia entre las medias (0.005 de mAP) es menor que la dispersión entre semillas con el backbone congelado (0.015). El rango del descongelado es más estrecho (0.004), pero con tres semillas por configuración no permite afirmar que sea más estable.

La figura 5 muestra sobreajuste en cuatro de las seis corridas: el mAP de validación alcanza su máximo entre las épocas 5 y 13 y cae entre 0.09 y 0.16 hacia el final. Los 441 fotogramas de entrenamiento provienen de solo 12 videos, por lo que el modelo los memoriza rápido. Las dos corridas estables son las de semilla 2, una con cada backbone, y el modelo final alcanzó su mejor época en la 46. Como cada corrida conserva su mejor época de validación, los resultados de prueba no se ven afectados (entre 0.637 y 0.653 de mAP). Por eso conviene seleccionar la época en validación en lugar de fijarla de antemano.

La escala de la confianza también depende del entrenamiento. Con el backbone congelado, la semilla 2 emite valores de hasta 0.99 y la semilla 1 no supera 0.25, aunque ambas alcanzan un F1 de 0.87 en validación con su propio umbral (0.82 y 0.16). Un umbral fijo funcionaría con una y fallaría con la otra; por eso se calibra por checkpoint. El mAP no se ve afectado, porque depende del orden de las predicciones y no de la escala de la confianza.

5.4 Latencia

Modelo	ms por imagen	Imágenes por segundo
RT-DETR ajustado	19.5	51
Grounding DINO, 3 <i>prompts</i>	471	2.1
Grounding DINO, 1 <i>prompt</i>	162	6

RT-DETR procesa 51 imágenes por segundo con una latencia estable (mediana de 19.1 ms, percentil 95 de 20.6 ms), ya que, sin NMS, el postprocesamiento no depende del número de cajas. En Grounding DINO, el *pipeline* ejecuta una pasada por *prompt* (verificado contando las pasadas del modelo), y la latencia incluye el NMS entre *prompts*. RT-DETR es 8 veces más rápido con un solo *prompt* y 24 veces con los tres usados; esta segunda cifra depende de la implementación, no de la arquitectura. Las mediciones se hicieron en una RTX 4070 SUPER con imágenes de 1920×1080. RT-DETR permite procesar video en tiempo real; Grounding DINO es útil solo para etiquetar *offline*. Esta diferencia justifica el diseño en dos etapas.

5.5 Origen del error: el tamaño de las cajas

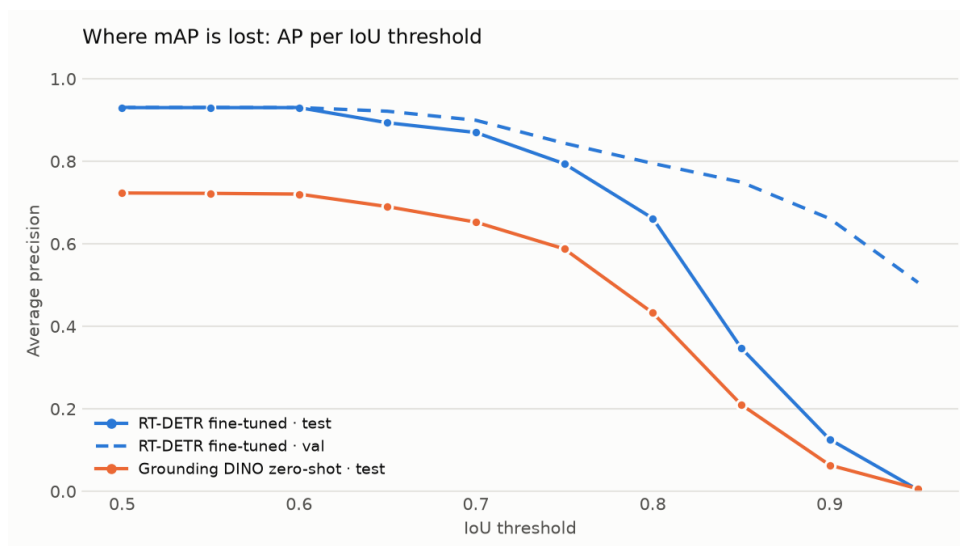


Figura 6. AP según el umbral de IoU para RT-DETR (prueba y validación) y Grounding DINO (prueba).

A partir de IoU 0.8, el AP de prueba cae en ambos modelos: a IoU 0.9 es de 0.13 en RT-DETR y de 0.06 en Grounding DINO, mientras que el de RT-DETR en validación se mantiene en 0.66 (figura 6). El patrón es sistemático. Las cajas de RT-DETR son un 5 % más angostas y un 7 % más bajas que las de prueba, con los cuatro bordes desplazados hacia adentro, igual que las pre-etiquetas de Grounding DINO (6 % y 9 %). Ambas cifras se miden sobre las cajas emparejadas con el *ground truth* (122 de RT-DETR y 90 de las pre-etiquetas), por lo que son poblaciones distintas. Esto es consistente con que el modelo haya aprendido el estilo de las pre-etiquetas: cerca de la mitad de las cajas de entrenamiento y validación son propuestas de Grounding DINO apenas corregidas, mientras que las de prueba se dibujaron a mano con un criterio algo más amplio.

De este resultado se desprenden tres lecturas:

- Es plausible que el techo del mAP a IoU alto lo fije la convención de etiquetado más que el modelo. Descongelar el backbone o agregar fotogramas no lo elevó, pero no se probó un modelo de mayor capacidad ni se reetiquetó una muestra para confirmarlo.
- El pre-etiquetado implica un riesgo poco visible: acelera la revisión, pero puede transferir al modelo entrenado el sesgo de localización del modelo *zero-shot*.

- La comparación entre ambos modelos sigue siendo justa, porque los dos comparten el sesgo. AP50 es la métrica que menos depende de la convención de las cajas.

5.6 Mapas de atención



Figura 7. De izquierda a derecha: una detección, la *self-attention* del encoder desde el token central de la caja y los puntos de muestreo del decoder.

En el decoder, los 96 puntos de muestreo de cada *query* se concentran sobre la valla detectada y sus bordes. En el encoder, la atención se midió sobre las 49 detecciones de prueba que aparecen en imágenes con dos o más vallas. La mediana de la atención que el token central dirige a las otras vallas es 1.63 veces la esperada por azar, y supera ese nivel en el 88 % de los casos; hacia la propia valla es 2.02 veces. Esto indica que la *self-attention* asocia regiones similares de toda la imagen, aunque sin una comparación con regiones de igual área que no sean vallas no puede atribuirse a una capacidad exclusiva del Transformer. El efecto, además, es moderado: parte de la atención se dispersa sobre el fondo.

5.7 Ejemplos

1. **Varias vallas a distintas distancias** (figura 8). RT-DETR detecta las tres; Grounding DINO agrega un falso positivo sobre un letrero comercial.
2. **Una valla aislada** (figura 9). Ambos modelos la detectan con una caja ajustada, RT-DETR con mayor confianza (0.99 frente a 0.82).
3. **Error compartido** (figura 10). Ambos modelos confunden la fachada de vidrio de un edificio con una valla.



Figura 8. Varias vallas a distintas distancias. RT-DETR a la izquierda y Grounding DINO a la derecha; en verde, el *ground truth*; en rojo, las predicciones con su confianza.



Figura 9. Una valla aislada, con la misma convención que la figura 8.



Figura 10. Error compartido: la fachada de vidrio de un edificio, con la misma convención que la figura 8.

5.8 Limitaciones

- **Conjunto de prueba pequeño.** Son 95 imágenes de 7 videos, y 52 provienen de dos de ellos. Las diferencias del orden de 0.01 de mAP no son concluyentes; la diferencia entre modelos sí lo es.
- **Una semilla por punto en la curva de aprendizaje.** Además, entre los dos primeros puntos cambió la partición de validación (de 65 a 95 imágenes).
- **Un solo anotador y una sola campaña de grabación.** No se midió el acuerdo entre anotadores ni la generalización a otras ciudades, horarios o cámaras.
- **Sin vallas pequeñas.** El conjunto de prueba no permite evaluar objetos pequeños o muy lejanos.
- **Latencia en GPU de escritorio.** No se midió en hardware embebido.
- **Validación con el estilo de las pre-etiquetas.** La época y el umbral se eligen sobre una validación cuyas cajas comparten ese estilo (a IoU 0.9, AP de 0.66 en validación frente a 0.13 en prueba), por lo que la selección lo favorece.
- **Punto de operación medido solo en validación.** En prueba no se calcularon la precisión y el *recall* con el umbral calibrado, ni intervalos de confianza.
- **Pocos ejemplos negativos.** Hay 5 imágenes sin vallas en todo el conjunto; no se midió la tasa de falsos positivos en video continuo, que es el uso previsto.
- **Hipótesis del sesgo de cajas sin verificar.** No se hizo una intervención (por ejemplo, reetiquetar una muestra) que confirme su causa.

6. Conclusiones y mejoras futuras

Conclusiones

1. **El pre-etiquetado *zero-shot* hace viable el proyecto.** Permitió etiquetar 631 imágenes en el tiempo disponible. Con los videos disponibles, sumar fotogramas deja de mejorar a partir de unas 200 imágenes.
2. **El ajuste fino supera al modelo *zero-shot* en precisión y, con la configuración usada, en velocidad,** lo que permite procesar video en tiempo real.
3. **La evaluación debe ser independiente del pre-etiquetado.** Un conjunto de prueba derivado de las mismas pre-etiquetas elevó el mAP de Grounding DINO de 0.481 a 0.677.
4. **El pre-etiquetado puede introducir un sesgo de localización.** El análisis sugiere que el modelo reproduce el tamaño de caja de las pre-etiquetas y que eso limita el mAP a IoU alto; confirmarlo requiere reetiquetar una muestra.
5. **En datos extraídos de video, la partición debe hacerse por video.** Una partición por imagen dejaría pasar a la prueba fotogramas casi idénticos a los de entrenamiento.
6. **Descongelar el backbone no produjo una diferencia atribuible.** Con tres semillas por configuración, reportar la variación entre ellas evitó sobreinterpretar diferencias pequeñas.

Mejoras futuras

- **Verificar y corregir el estilo de las cajas.** Reetiquetar a mano una muestra de entrenamiento y validación para confirmar la hipótesis del sesgo y, si se confirma, corregirlo; nunca con la prueba.
- **Completar la evaluación.** Precisión, *recall* y F1 en prueba con el umbral calibrado e intervalos *bootstrap* por video; tasa de falsos positivos en video continuo; latencia de Grounding DINO en una sola pasada; un backbone de mayor capacidad.
- **Aumentar la diversidad de escenas.** Sumar otras ciudades, horarios y condiciones climáticas, ya que más fotogramas de los mismos videos no aportan.
- **Controlar el sobreajuste.** Usar *early stopping* con paciencia y una regularización más fuerte.
- **Validación cruzada agrupada por video.** Permitiría estimar mejor la varianza.
- **Calibración de la confianza.** Por ejemplo, *temperature scaling*.
- **Despliegue embebido.** Exportación a ONNX/TensorRT en FP16 y seguimiento entre fotogramas para contar vallas únicas en un recorrido.
- **Subclases para auditoría.** Valla tradicional, pantalla digital y valla vacía.

7. Planificación del equipo

El proyecto se realizó de forma individual.

Tarea	Responsable	Estado
Construcción del conjunto de datos (pre-etiquetado, revisión y prueba desde cero)	Christian Conchari	Completada
Entrenamiento, evaluación y análisis de resultados	Christian Conchari	Completada
Repositorio, <i>pipeline</i> , pruebas y documentación	Christian Conchari	Completada
Informe técnico	Christian Conchari	Completada
Presentación final (clase 8)	Christian Conchari	Pendiente

Referencias

- Carion, N. et al. (2020). End-to-End Object Detection with Transformers. *ECCV*.
- Dosovitskiy, A. et al. (2021). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *ICLR*.
- Lin, T.-Y. et al. (2014). Microsoft COCO: Common Objects in Context. *ECCV*.
- Liu, S. et al. (2024). Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection. *ECCV*.
- Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS*.
- Zhao, Y. et al. (2024). DETRs Beat YOLOs on Real-time Object Detection. *CVPR*. Versión preliminar: arXiv:2304.08069v2 (2023).
- Zhu, H., Chen, B. y Yang, C. (2023). Understanding Why ViT Trains Badly on Small Datasets: An Intuitive Perspective. *arXiv:2302.03751*.
- Zhu, X. et al. (2021). Deformable DETR: Deformable Transformers for End-to-End Object Detection. *ICLR*.